

Dirty Spokes Dirtbike Rental Management System

Step 3 Draft

Web App URL: <http://classwork.engr.oregonstate.edu:41801/>

a) Project Outline, DB Outline, ERD Schema, & Sample Data Updated Version:

Overview

Dirty Spokes is a recreational dirtbike rental facility and track located in sunny Southern California, operating a fleet of 50 bikes across 8 major brands including Honda, Yamaha, and Kawasaki. The facility services roughly 500 unique customers per year across an estimated 2,500 annual rental sessions. Currently, staff manage bike inventory, customer records, and rental history through manual bookkeeping and paper waivers.

Over the past year, this has led to a number of operational issues including customer records being lost or duplicated, no reliable way to track when a customer had which bike at any given time, and difficulty identifying when a bike was last rented or how many hours it has accumulated.

A database-driven web application will allow Dirty Spokes administrators to manage Bikes, Brands, Customers, and Rentals in a unified system, ensuring accurate rental records, up-to-date usage metrics for bike maintenance and popularity, and smoother day-to-day operations at the track.

In applying the normalization process to this design, no changes to the outline or ERD are required after Step 1's feedback fixes. The schema is in 3NF. All attributes contain atomic values with no repeating groups, all non-key attributes are fully dependent on their primary keys with no partial dependencies, and no non-key attribute determines another non-key attribute.

Foreign key ON DELETE and ON UPDATE behaviors have been documented in Step 3 for clarity on relationships in the database outline.

Database Outline

Bikes: records the dirtbikes available in the rental fleet

- bikeID: INT UNSIGNED, AUTO_INCREMENT, unique, NOT NULL, PK
- frameNumber: VARCHAR(50), unique, NOT NULL
- brandID: INT UNSIGNED, NOT NULL, FK → Brands.brandID
- modelName: VARCHAR(100), NOT NULL
- engineSize: VARCHAR(10), NOT NULL - e.g. "250cc", "450cc"
- bikeYear: INT UNSIGNED, NOT NULL
- engineHourMeter: DECIMAL(6,1), NOT NULL
- Relationships:
 - M:1 between Bikes and Brands, many bikes can belong to one brand implemented by FK brandID. ON DELETE RESTRICT prevents a brand from being deleted while bikes are associated with it. ON UPDATE CASCADE propagates brand ID changes to all associated bikes.
 - 1:M between Bikes and Rentals, one bike can appear in many rental records implemented by FK bikeID in Rentals

Brands: records the manufacturers of the bikes in the fleet

- brandID: INT UNSIGNED, AUTO_INCREMENT, unique, NOT NULL, PK
- brandName: VARCHAR(100), unique, NOT NULL
- countryOfOrigin: VARCHAR(100), NOT NULL
- localDealer: VARCHAR(255), NOT NULL
- Relationships:
 - 1:M between Brands and Bikes, one brand can have many bikes implemented by brandID FK in Bikes

Customers: records customers who rent bikes at the track

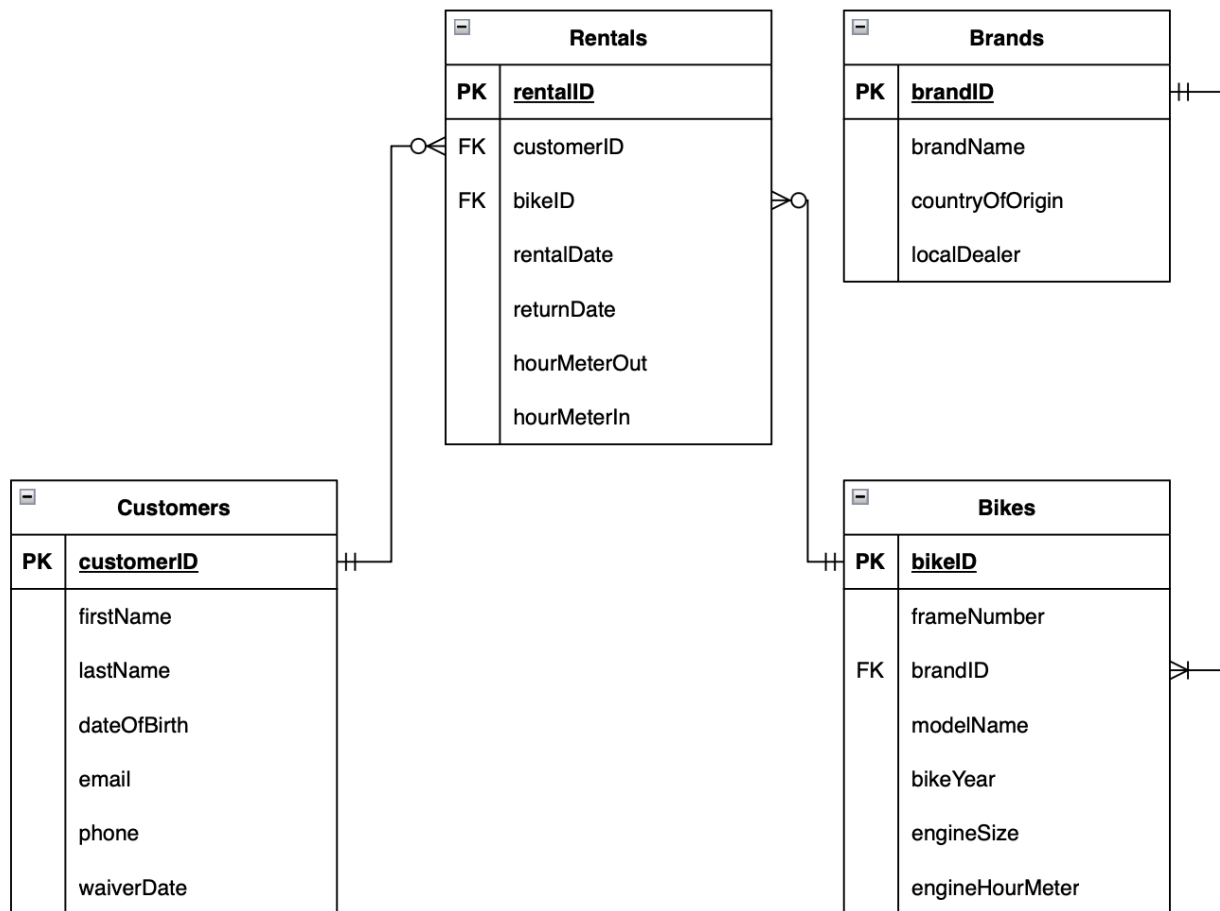
- customerID: INT UNSIGNED, AUTO_INCREMENT, unique, NOT NULL, PK
- firstName: VARCHAR(50), NOT NULL
- lastName: VARCHAR(50), NOT NULL
- dateOfBirth: date, NOT NULL
- email: VARCHAR(255), unique, NOT NULL
- phone: VARCHAR(20), NOT NULL
- waiverDate: date, NOT NULL
- Relationships:
 - 1:M between Customers and Rentals, one customer can have many rental records implemented by FK customerID in Rentals

Rentals: intersection table recording which customer rented which bike and when

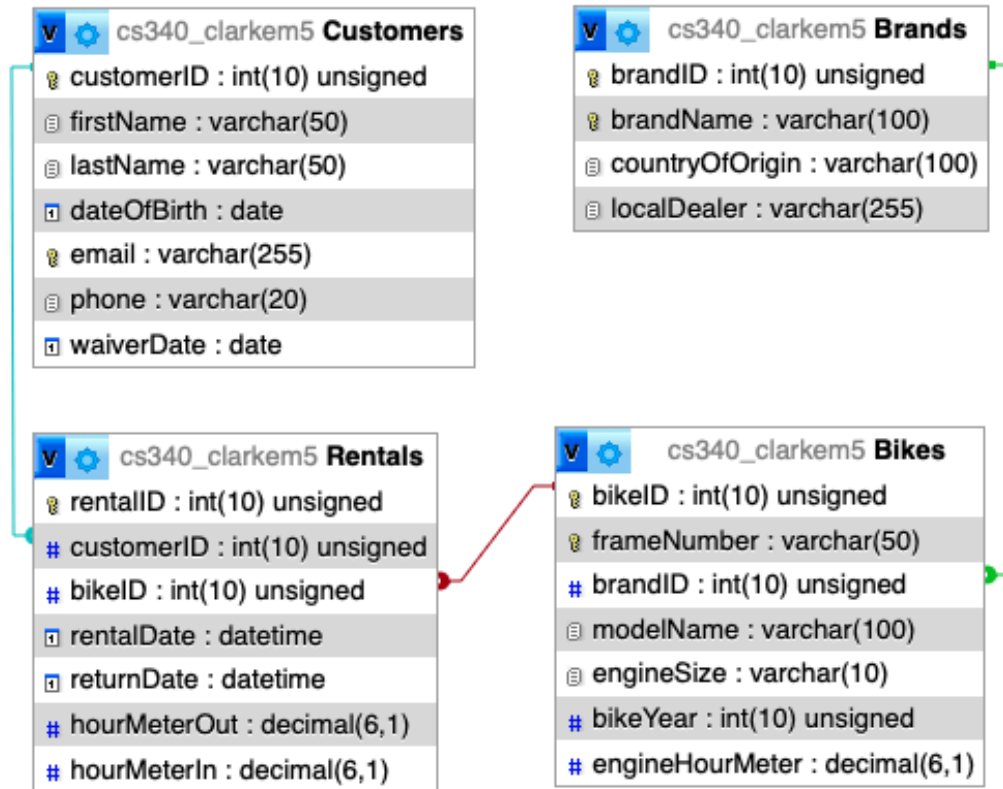
- rentalID: INT UNSIGNED, AUTO_INCREMENT, unique, NOT NULL, PK
- customerID: INT UNSIGNED, NOT NULL, FK → Customers.customerID
- bikeID: INT UNSIGNED, NOT NULL, FK → Bikes.bikeID
- rentalDate: datetime, NOT NULL

- returnDate: datetime, NOT NULL
- hourMeterOut: DECIMAL(6,1), NOT NULL
- hourMeterIn: DECIMAL(6,1), NOT NULL
- Relationships:
 - M:1 between Rentals and Customers, many rental records can belong to one customer where Customers.customerID is PK and Rentals.customerID is FK. ON DELETE RESTRICT prevents deletion of a customer with existing rental records. ON UPDATE CASCADE propagates ID changes.
 - M:1 between Rentals and Bikes, many rental records can reference one bike where Bikes.bikeID is PK and Rentals.bikeID is FK. ON DELETE RESTRICT prevents deletion of a bike with existing rental records. ON UPDATE CASCADE propagates ID changes.
 - M:N between Customers and Bikes, a customer can rent many bikes and a bike can be rented by many customers

Entity-Relationship Diagram



Schema Diagram



Example Data

Bikes Table

bikeID	frameNumber	brandID	modelName	engineSize	bikeYear	engineHourMeter
1	JYACB11C0RA015607	3	YZ65	65cc	2024	15.8
2	VBKXWM236MM321033	2	300 XC-W	293cc	2021	71.4
3	VNBS648C4PB000307	1	SE-F 500 Factory 4T	478cc	2023	225.5

Brands Table

brandID	brandName	countryOfOrigin	localDealer
1	Sherco	France	Trials Offroad
2	KTM	Austria	SoCal Moto
3	Yamaha	Japan	SoCal Moto

Customers Table

customerID	firstName	lastName	dateOfBirth	email	phone	waiverDate
1	Jeremy	Stevens	1989-06-11	jsteve0@email.com	555-0101	2024-11-22
2	Andrea	Hernandez	2003-11-25	andrea44@email.com	555-0202	2025-04-17
3	Roman	Lutoslawski	1978-08-26	roman444@email.com	555-0303	2020-08-20

Rentals Table

rentalID	customerID	bikeID	rentalDate	returnDate	hourMeterOut	hourMeterIn
1	3	3	2025-04-06 11:22:56	2025-04-08 16:43:16	219.9	225.5
2	3	2	2026-04-10 09:30:17	2026-04-13 10:17:44	60.6	68.0
3	2	2	2026-04-19 10:19:19	2026-04-19 15:22:04	68.1	71.4

b) Fixes based on Feedback from Previous Steps:

Step 1 Accepted Fixes

- Add lengths to VARCHAR and INT attributes - VARCHAR lengths, INT signs, and DECIMAL placing have been added to all attributes throughout the Database Outline for greater precision and clarity.
- Make Brand mandatory to Bikes - The Brands→Bikes relationship in the ERD has been updated to reflect a mandatory relationship, as every bike in the fleet must be associated with a brand.
- Remove M:N connector between Customers and Bikes in ERD - The direct line between Customers and Bikes has been removed from the ERD. This relationship is already properly represented through the Rentals intersection table and the direct connector was redundant.
- Add cardinality to connections in the ERD - Proper cardinality markers have been added to all relationship lines in the ERD.
- Edit engineHours to singular - engineHours was not present in the Database Outline so it has been added to the Bikes entity and changed to a singular name, engineHourMeter.

Step 1 Declined Suggestions

- Add Employees table - The scope of this system is intentionally limited to bike inventory, customer records, and rental history. This is an administrator-facing database tool, not an employee management system. Adding an Employee table would exceed the intended scope of the project.
- phone attribute changed from VARCHAR to INT - We chose to keep phone as VARCHAR since phone numbers may include formatting characters such as dashes,

parentheses, spaces, and country code prefixes. Storing phone as INT would risk data loss and overflow on numbers with country codes as well as prevent storing any non-numeric character formatting.

- Make localDealer optional - This suggestion was not employed as recording the nearest outside facility that sells or services a specific brand is a necessity in assembling a fleet of popular models and maintaining them throughout their lifecycle. The scenario of a major brand ceasing operations and no longer being available is highly unlikely, and the discontinuation of a brand would result in its removal from the active fleet entirely rather than keeping it with a null dealer field.
- Add license and insurance attributes to Customers - This suggestion is not a necessity in this context. 50cc MX bikes and increasingly popular electric bikes of comparable sizes are geared toward riders as young as 5 years old who would not have a license or insurance to record. Beyond this, off-highway vehicles do not require a license or insurance to operate in legal settings such as a designated MX or enduro track. Adding these fields would therefore not be applicable to a significant portion of the customer base and is out of scope for the current system design.

Step 2 Feedback

No grader feedback or peer review comments were received on Ed Discussion for Step 2. No other changes were made as a result.

Citations

The web application scaffolding for Step 3 was written with the help of Claude (Anthropic). Prompts described our database schema and requested UI scaffolding templates for a Node.js/Express/Handlebars application.

All other work in this document was done without the use of AI tools.